Course: BCS407 – Artificial Intelligence

Project Report

Project Title: Computer Vision Object Detection Project

Subtitle: AI-Based Smart Campus Detection System

Theme 3: Campus Safety Monitoring (Wet Floor Signs, Fire Alarms, Emergency Exits, Safety Equipment)

Name	ID
Leanne Jessica Rodrigo	20210001983
Hassan Mujtaba	20220002085
Khaled Riyan	20220001526
Izaan Shaikh	20220001735

Instructor: Dr Najla Al Futaisi

Date of Submission: 30th April 2026

1. Introduction

In today's educational institutions, campus safety is an increasing concern as the ability to rapidly detect risks and enforce safety procedures can save lives. In settings such as Canadian University Dubai, different physical safety signs are installed across the campus - fire alarms, exit signs, safety signs and workers wearing helmets - but it is impossible to manage these at a large scale.

In this project, we have created an AI-Based Smart Campus Safety Detection System.The objective is to implement an object detection system using deep learning techniques that can detect and classify important safety objects on and off the campus. Our system aims to detect 15 classes of objects, such as fire alarms, directional emergency exit signs, blocked exits, helmets, persons and signage, representing a wide range of campus safety situations.

In this project, we gathered and annotated a dataset of 1,672 images with Roboflow, and trained a YOLOv8m model on them. Our model runs in real time, drawing boxes around objects and displaying labels with associated probabilities. Once deployed, this system could be integrated with a CCTV network to detect safety infringements or risks in real time.

This project demonstrates the full machine learning workflow: data collection, annotation, model selection, training, evaluation, and inference (real-time deployment). In this report, we describe each step, explain our design choices, discuss the ethical implications, and report evaluation of model performance using computer vision metrics. Our model reached a mAP@0.5 of 97.8%, showing that YOLOv8 performs well in this application.

2. Literature Review

Object detection is a fast-developing area of computer vision, which is powered by the advances in deep learning and the availability of large-scale annotated data. Earlier methods such as Histogram of Oriented Gradients (HOG) combined with Support Vector Machines (SVM) paved the way for the current approaches(Dalal & Triggs, 2005). The next leap was with the introduction of region-based convolutional neural networks which demonstrated the importance of deep features for object detection(Girshick et al., 2014). Later techniques addressed the computational inefficiency using a reuse of convolutional features and region proposals, enabling almost end-to-end training(Ren et al., 2015).

The one-shot detector models such as SSD and YOLO focused on speed by performing bounding box and classification predictions in one go(Liu et al., 2016; Redmon et al., 2016). The model used in this research, YOLOv8, is a new addition to the Ultralytics family.This detector uses anchorless detection, decoupled heads and loss functions (Jocher et al., 2023).

YOLO detectors have been used in safety monitoring in several studies. These techniques have been applied in different scenarios, including safety monitoring in construction, to accurately detect helmets and vests(Jiang et al., 2021).Other studies have proposed real-time detectors of hazards such as fire alarms, showing they can be run on edge devices(Li et al., 2022).More recently, transformers have also been proposed for detections, but they are generally more computationally intensive and not suitable for edge devices(Carion et al., 2020).

This paper builds on this work, by adapting the YOLOv8 model to a multi-class campus safety hazard detector. This approach can detect 15 classes of safety hazards, and has more categories than other works. This makes it more likely to be encountered in a campus environment, and it's suitable as a comprehensive monitoring system.

3. Dataset

Collection and Composition

Our dataset was created and organised using Roboflow, and comprises 1,672 images across 15 object classes of interest for campus safety. Our dataset was exported in YOLOv8 format. Those 15 classes are: Fire alarm, Left Exit, Left/Right Exit, Right Exit, Straight Exit, blocked emergency exit, boxes blocking exit, emergency exit door, exit block violation, exit-sign, head, helmet, objects blocked emergency exit, person and sign.

The dataset includes images from various real-world settings, such as indoor hallways, construction sites, offices and public buildings. The images were taken under varying lighting conditions (daylight, fluorescent indoor lighting, low light), and from different distances and angles. This was done to ensure the model would be generalisable to different environments.

This decision to identify fifteen unique safety violations as opposed to generalizing these types of hazards into broad classifications was a calculated choice. It is beneficial when the results are actionable. A safety officer who receives an alert for an issue with exits will want to know if it is due to boxes blocking the exit or if it is because there is a structural obstruction. Likewise, a safety officer will need to know if the worker has lost their helmet or is simply turning his/her back to the camera. Outputs produced using coarser levels of categorization will be less descriptive and thus provide fewer options for action. There were costs associated with the decision to use 15 different categories. These categories required much more annotation data per category to achieve reliable object detection. Many of the minority categories (for example, 5 examples of exit blockage violation and 6 examples of blocked emergency exit) did not have enough examples for the model to develop consistent image features. Similarities in appearance among categories made competing for the model's attention difficult. Four directional exit signs (Left, Right, Left/Right, and Straight) had very similar ISO 7010 iconography; they differed only in terms of the direction of the arrow. At lower resolutions or angles, detecting the differences may become increasingly difficult. The confusion matrix demonstrates how challenging this is, since only boxes blocking the exit had the lowest score of 56% . If we had combined these four categories into one category labeled "exit sign," we would likely see significantly better classification rates within each category—but this would reduce our ability to direct someone towards the closest exit during an emergency.

Annotation Process

We used Roboflow’s labelling tool to create annotations. All images were checked and "boxes" were added for every instance of every relevant class. Special attention was paid to differentiate between visually similar classes, such as specific exit signs and between 'emergency exit blocked' and 'boxes blocking exit'. We did not apply any pre-processing or augmentation to the images at the time of export; it was applied during model training via YOLOv8 (mosaic, random flip, colour jitter, scale variation) which allowed us to have more control and prevented data leakage. The images below display example predictions made by the trained model on actual images, illustrating a range of the categories our system is capable of identifying:

Figure 1: Wet floor sign (conf. 0.94)

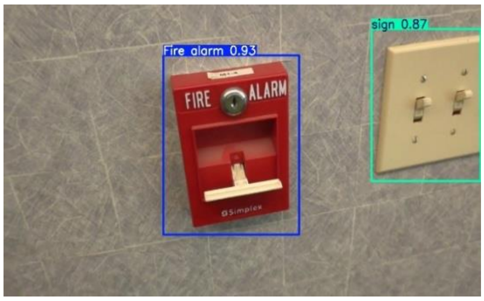


Figure 2: Fire alarm detected (conf. 0.93)

Figure 3: Exit-sign detected (conf. 0.90)

Figure 4: Objects blocking exit (conf. 0.93)

Figure 5 : Blocked emergency exit (conf. 0.92)

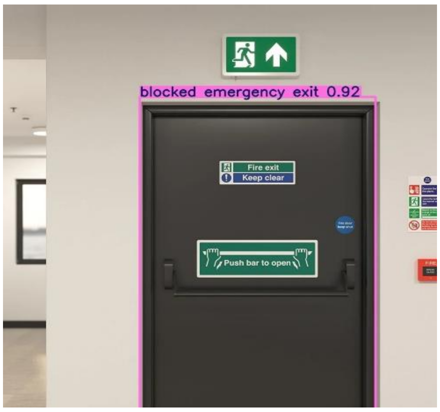


Figure 6 : Emergency exit door + Fire alarm

Figure 7: Wet floor signs detected (0.73–0.94)

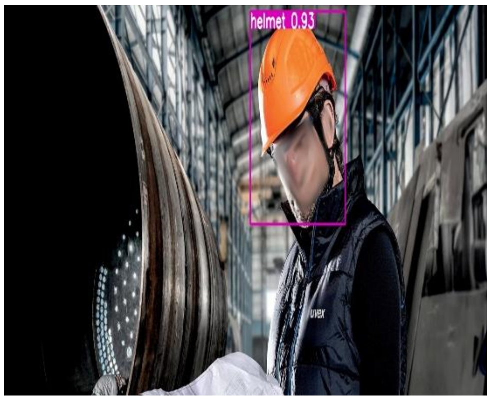


Figure 8: Helmet detected (conf. 0.93)

Figure 9 : Head detections in warehouse setting

Figure 1–9: Example model forecasts across labeled class categories. Scores of confidence displayed for each detection

Training Batches

The images below are a selection of example labeled images from three batches of training, showcasing the variation in scene and class types, as well as annotation techniques.

Figure 10: Training batch 0 - examples with bounding boxes around some of the safety-related classes (helmets, fire alarms, people, exit signs).

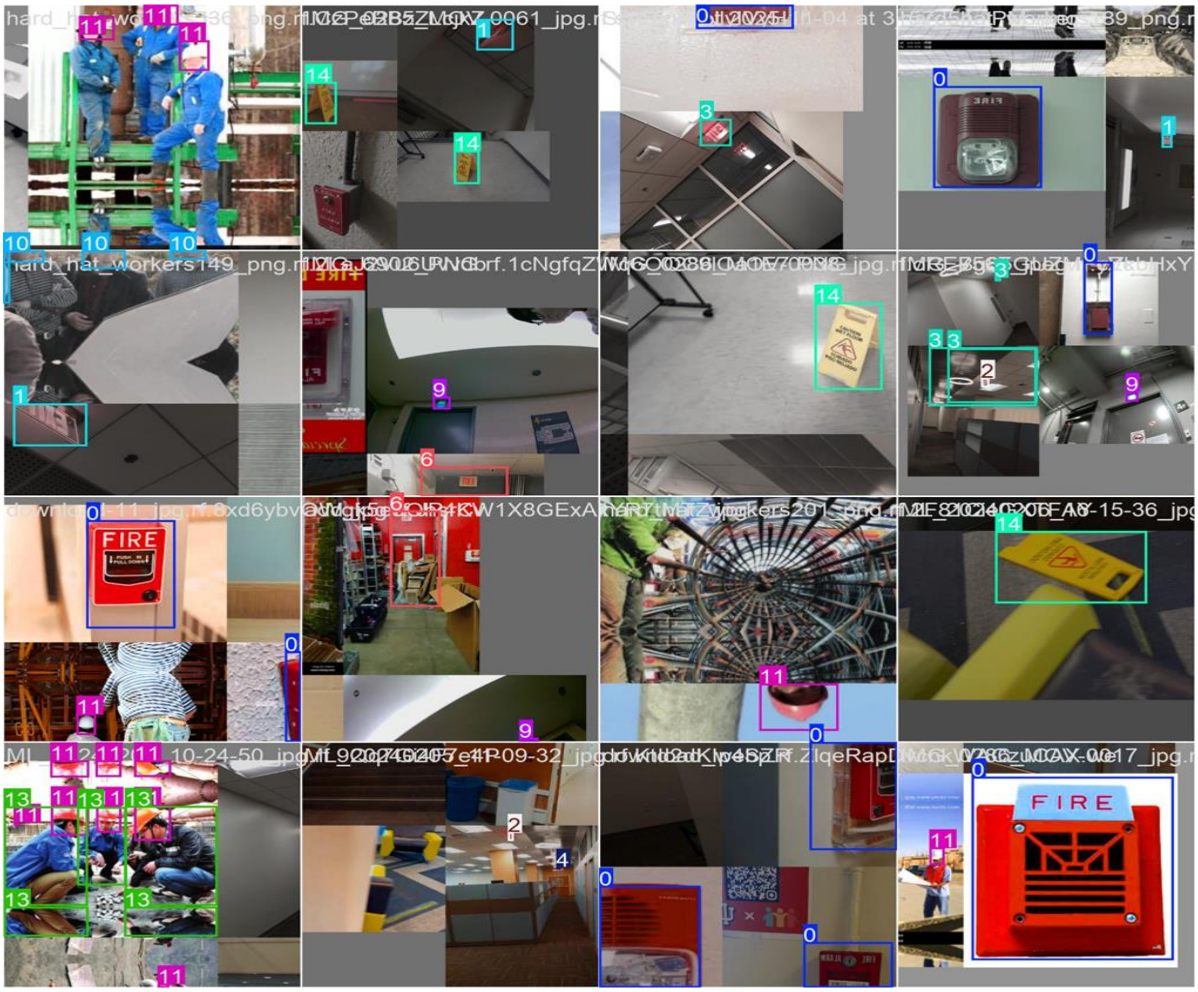
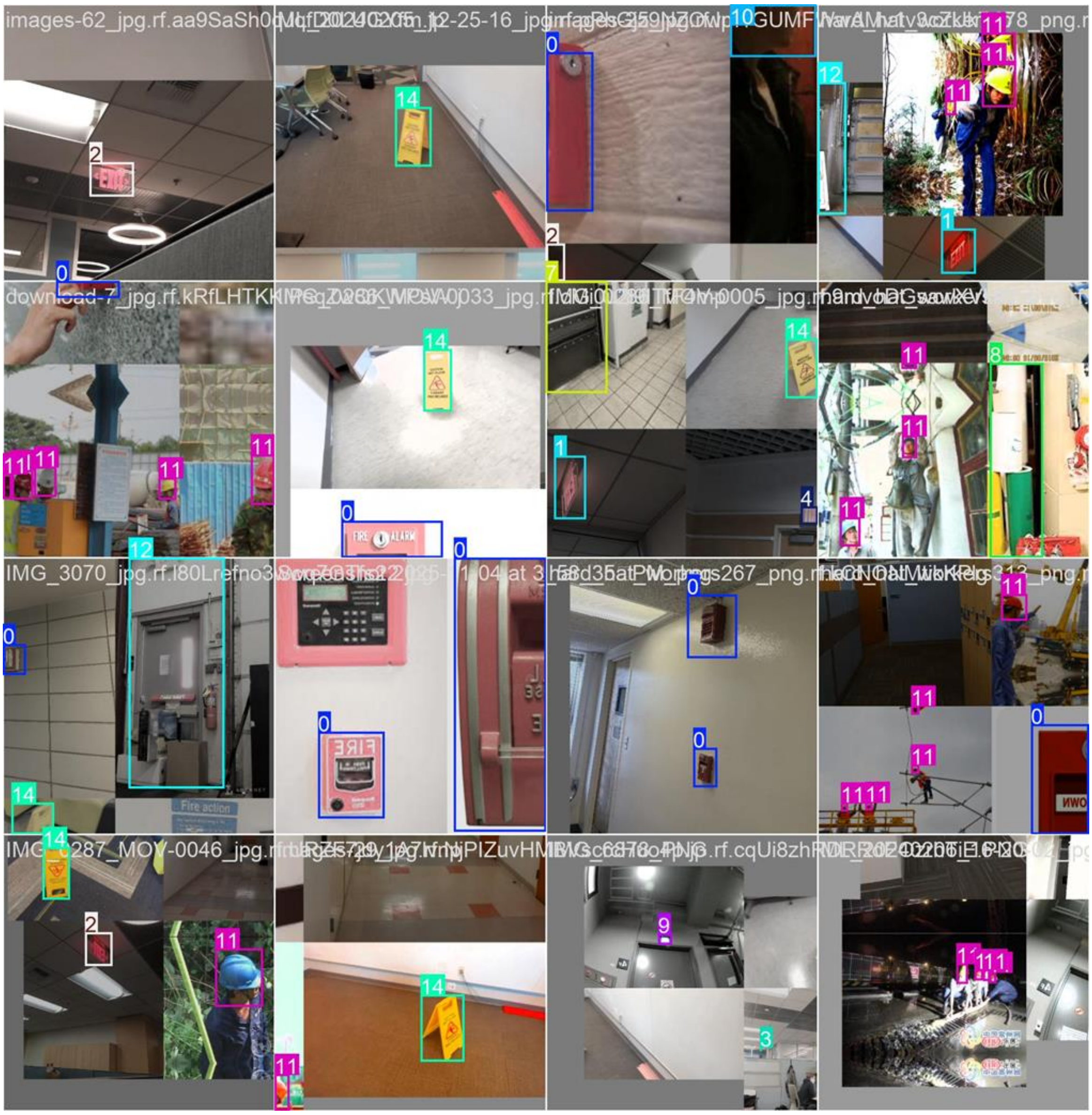


Figure 11: Training batch 1 - additional annotated samples showing varied indoor/outdoor scenes in our dataset.

Figure 12: Training batch 2 - annotated samples from subsequent training batches featuring different lighting and perspectives.



4. Methodology

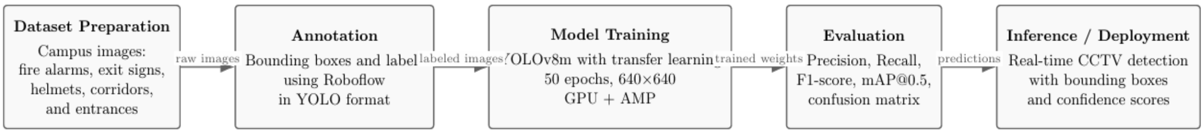


Figure 13: Model's Pipeline

Our system adopts an artificial intelligence pipeline, as shown in Figure 3. The process involves five stages: data preparation, annotation, training, evaluation and inference.

Preparation of the Dataset:

In the first stage, we gather images of different campus safety scenarios such as fire alarms, exit signs, helmets and blocked exits. We collected 1,672 images from a variety of settings including hallways, construction sites and public spaces to achieve variability in context, lighting, angle and scale.

The Annotation Process:

We annotated all images in Roboflow. Each instance was given a bounding box in YOLO format. Particular attention was paid to separating visually similar classes, like "blocked emergency exit" and "boxes blocking exit", to enhance class separation during model training.

Training of the Model:

A YOLOv8m model was trained with the annotated training data. We used transfer learning with pre-trained weights (yolov8m.pt) to enable the model to benefit from learned features. The model was trained for 50 epochs with a batch size of 32 and image size of 640×640. Training automatically applied data augmentation strategies including mosaic augmentation, random flips and scaling.

The Evaluation Stage:

The model was evaluated after training using common object detection evaluation metrics such as Precision, Recall, F1-score and mean Average Precision (mAP@0.5). Confusion matrices, Precision-Recall curves and F1-confidence curves were also used to understand the performance (both overall and per class).

The Inference (Deployment) Stage:

Finally, the model (best.pt) was tested on new images. It takes an image as input and produces bounding boxes, labels and confidence scores. This phase represents the deployment phase, where the model can be deployed with CCTV systems to identify safety issues in real time.

Model Selection

In selecting our object detector for this project, we chose YOLOv8 (the YOLOv8m architecture). The YOLOv8m version was chosen to offer a better compromise between detection precision and computational efficiency in comparison to smaller versions like YOLOv8s. Our decision was guided by the following:

•Speed: YOLOv8 is a one-shot detector that performs all operations during a forward propagation through the model, thereby making it suited to perform inference tasks in real time, e.g., real-time surveillance cameras deployed within the campus setting.

•Moderate scale: On standard benchmarks, YOLOv8 outperforms SSD in terms of accuracy and is considerably faster than any two-stage detectors such as Faster R-CNN.

•Framework support: Ultralytics comes with good Python support that includes training utilities, built-in training and validation metrics, and models exporters.

•Pre-trained models: The yolov8m.pt pre-trained models come with transferred knowledge from larger data sets, thereby making it possible for the models to learn faster from custom data sets.

As a competing alternative, we also considered Faster R-CNN. This model can provide slightly better accuracy on small-scale objects; however, it runs at 5–10 times slower during inference.

Training Configuration

The model was trained based on the following parameters as specified in train.py:

Parameter	**Value**
Base Model	YOLOv8m (yolov8m.pt)
Epochs	50

Image Dimension	640 x 640 pixels
Batch Size	32
Device	GPU(device=0)
Workers	8
Cache True	(Catching in RAM to load epoch faster)
AMP	True (Mixed Precision Training)
Data Source	Data.yaml (15 classes; 1672 images

Mixed-precision training (AMP) was turned on to lower GPU memory consumption and boost training performance without compromising the accuracy of the output model. RAM caching was employed to make the process of loading data during training quicker, which proves highly efficient when working with such a large-scale dataset. The best-performing model checkpoint was automatically selected based on the maximum value of mAP@0.5 reached during validation.

Inference process

Inference was executed using the predict.py script after the training process was completed. The best checkpoint (best.pt) is loaded from the model directory and used to detect objects within a target image at the confidence threshold of 0.5. The output images are saved into runs/detect/predict/ along with annotated bounding boxes, class labels, and corresponding confidence levels.

5. Results

Results and Evaluation

We trained the YOLOv8m model for 50 epochs using 1,672 images and 15 safety-related classes. The model was trained on a group member’s PC with an NVIDIA GeForce RTX 3070 Ti laptop GPU, using CUDA and Automatic Mixed Precision (AMP).

Training Progress

Losses were consistently decreasing over the 50 epochs. The bounding box loss dropped from 1.487 at epoch 1 to 0.738 at epoch 50, classification loss fell from 2.547 to 0.365, and DFL loss reduced from 1.432 to 0.979. Validation losses also continued to decrease without a gap, indicating that the model was not overfitting and had good generalization as shown in figure 14.

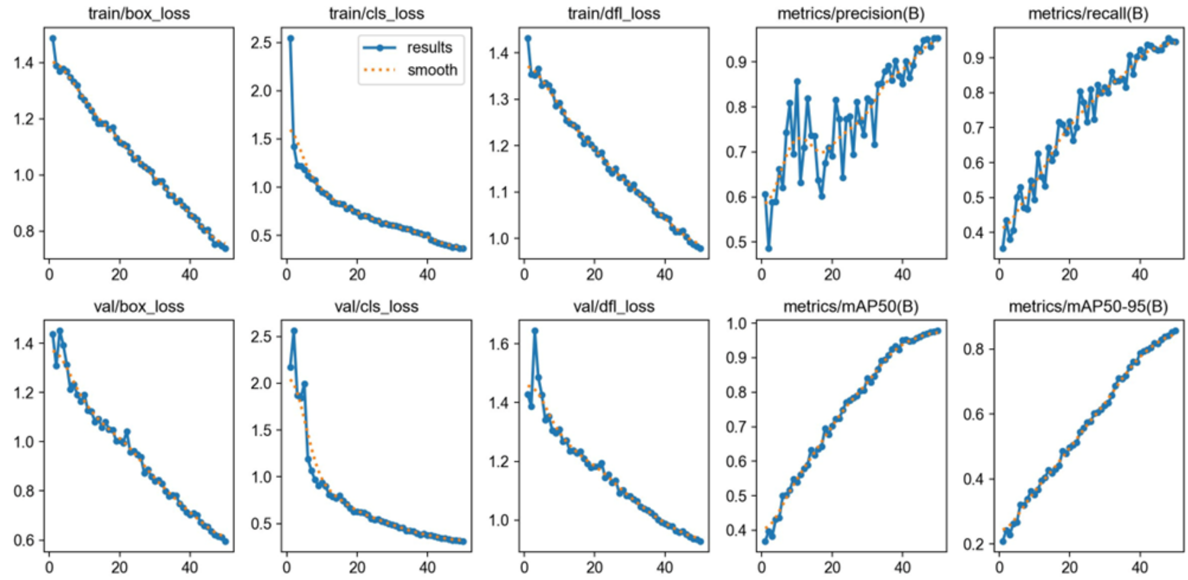


Figure 14: Losses and evaluation metrics for training and validation during 50 epochs of training.

Detection Accuracy

At the final epoch, the model achieved the following metrics:

Metric	Value
Precision	0.953
Recall	0.946
mAP@0.5	0.978
mAP@0.5:0.95	0.856

The mAP@0.5 of 97.8% shows exceptional detection accuracy for all 15 classes. The Precision-Recall (PR) curve shows that the AP of most classes was greater than 0.93, with blocked emergency exit, emergency exit door, exit sign, and sign all reaching 0.995.

Confusion Matrix Analysis

The normalized confusion matrix shows good class-specific performance. The fire alarm was 99% accurate; left exit and left/right exit were 97% accurate; and blocked emergency exit, emergency exit door, exit sign, and sign were 100% accurate. The helmet was correctly detected at 98% and the head at 99%, which is important for helmet compliance. This type of performance is further exemplified through figures 15 and 16.

The lowest accuracy was boxes blocking the exit at 62%, which was sometimes confused with objects blocking the emergency exit (44%). This is understandable due to the visual similarities between various scenarios of blocked exits. The objects blocked emergency exit class itself achieved 56%, also due to limited samples in the training set—only 18 instances in the dataset (see class distribution below).

Figure 15: Confusion Matrix – actual counts of true versus predicted class labels for all 15 categories.

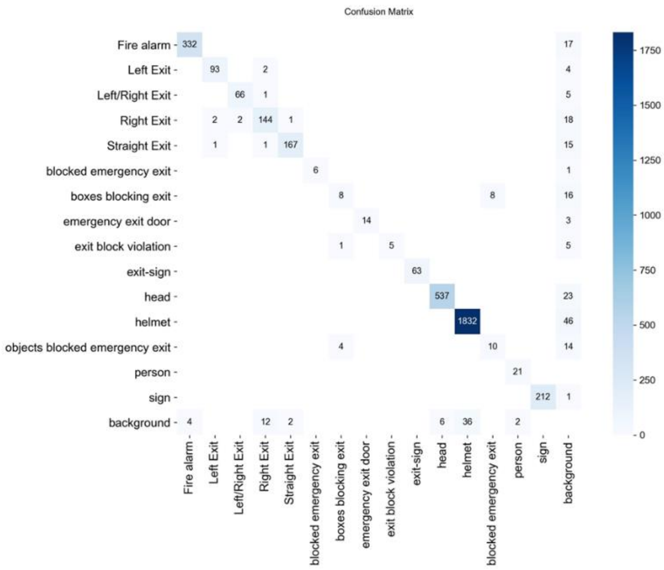
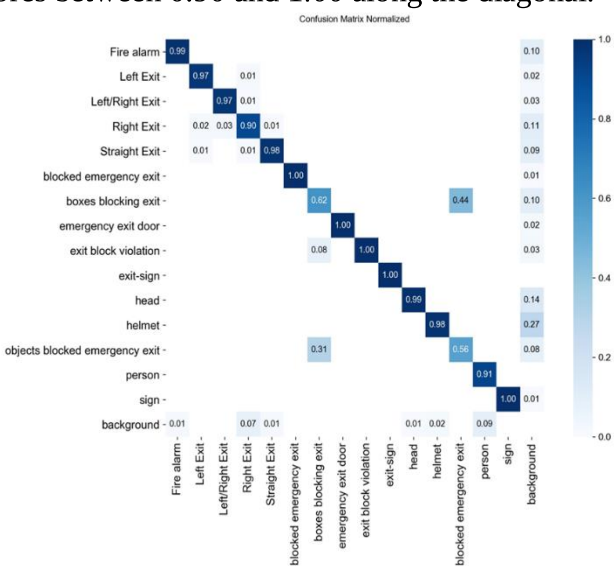


Figure 16: Normalized Confusion Matrix – relative precision for each category. The majority of classes achieve scores between 0.90 and 1.00 along the diagonal.

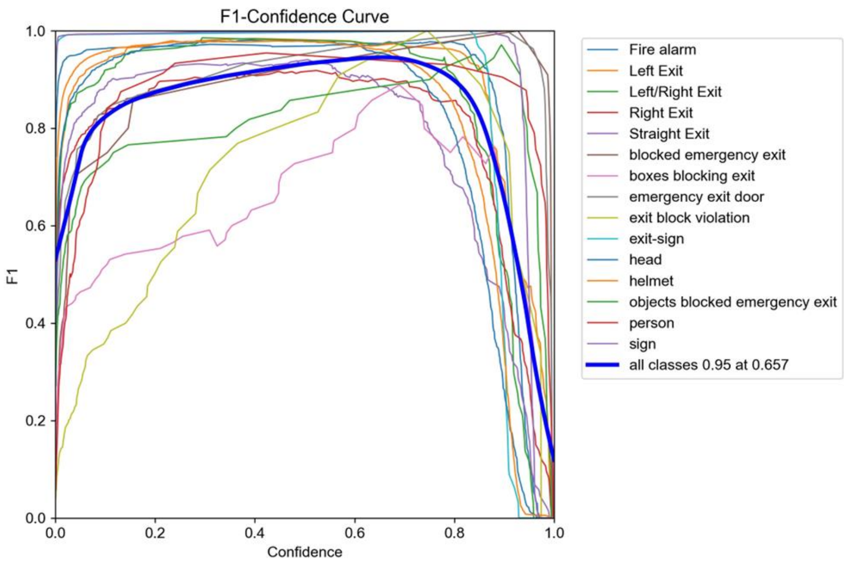


The normalized matrix highlights several important points. Fire Alarm gets 0.99, Left Exit gets 0.97, Left/Right Exit gets 0.97, blocked emergency exit gets 1.00, emergency exit door gets 1.00, exit block violation gets 1.00, exit-sign gets 1.00, head gets 0.99, helmet gets 0.98, and sign gets 1.00. The least accurate class is boxes blocking exit with 0.62. It has 44% misclassification to objects blocked emergency exit class, which looks similar. Objects blocked emergency exit class gets 0.56. They look visually very similar, hence the confusion. Person class gets 0.91, while Right Exit gets 0.90.

F1-Confidence Curve Analysis

Graph of F1-Confidence shows the variations in F1 scores at various confidence levels. An F1 score of 0.95 is obtained at the confidence level of 0.657. This means that a slight increase in the default confidence level of 0.5 would result in a higher deployment efficiency while having no significant effect on the recall rate.

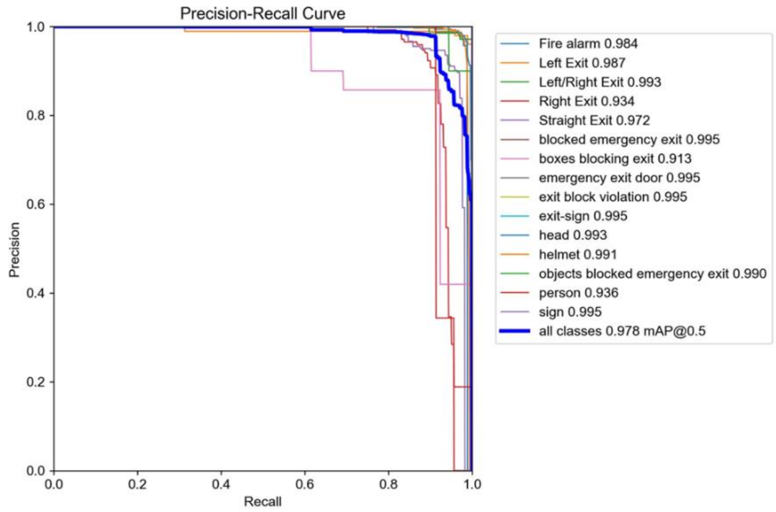
Figure 17: F1-Confidence Graph for all 15 categories. The prominent blue line indicates a macro-average peak F1 score of 0.95 at a confidence level of 0.657.



Precision-Recall Curve Analysis

Further validation of this comes from the Precision-Recall graph. The model exhibits an overall mAP@0.5 score of 0.978. The majority of the classes have their AP scores above 0.93, with many critical safety classes having their scores above 0.99. Shown below in Figure 18.

Figure 18: Precision-Recall Curve for all fifteen classes.Total mAP@0.5 = 0.978. The legend displays the AP values for each class.



Validation Batch Results

The validation batch predictions exhibit excellent correspondence with the true labels. It is able to correctly localize and classify objects under various lighting, scale, and angle conditions. Highly confident predictions are consistently seen in essential safety objects, such as helmets, fire alarms, and exits.

Figure 19: Growth truth labels for validation batch 0.

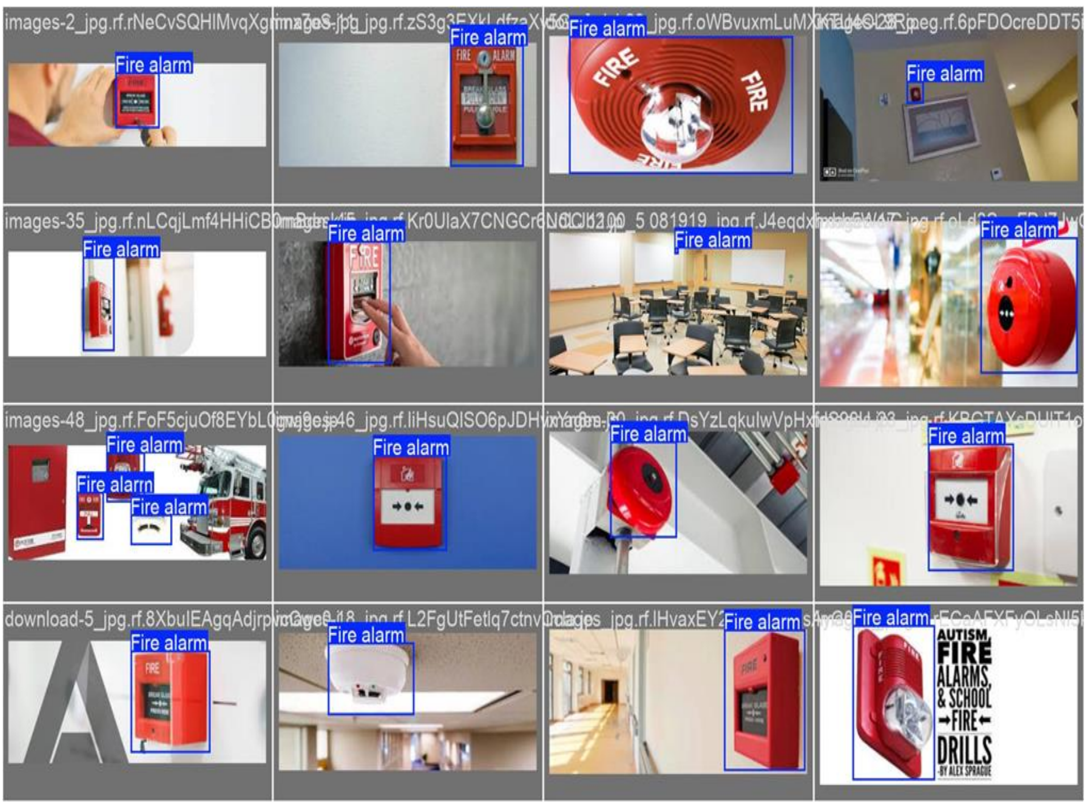
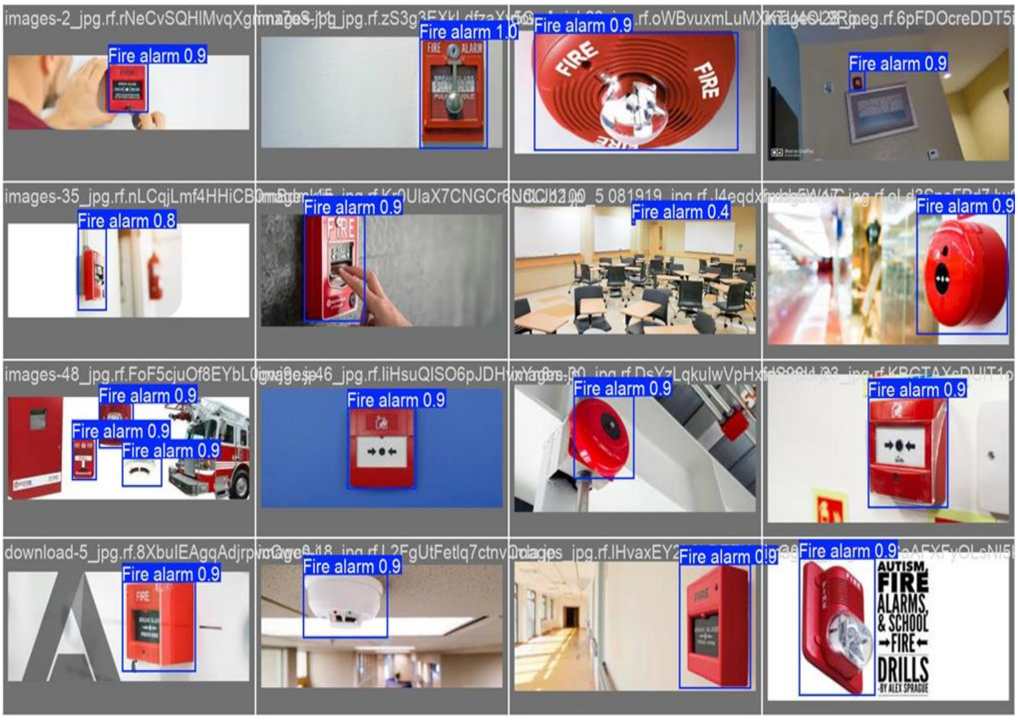


Figure 20: Validation batch 0 – predictions from the model. Bounding boxes and confidence scores closely align with the growth labels.



Dataset Distribution

The labels graph shows an imbalance. Helmet had 1,868 instances; head had 543. By comparison, there were 5 instances of exit block violation, 6 of blocked emergency exits, and 18 of objects blocking emergency exits. This is the main cause of the reduced performance in these classes.

Qualitative Results

Qualitative results on sample predictions on the validation set show correct bounding boxes and class labels in a variety of real-life situations, ranging from different lighting, sizes, and angles. Fire alarms and signs were detected with confidence scores of 0.9 or greater.

Summary

The model performs well and can be used for safety monitoring. The mAP and F1 scores indicate successful helmet, head, exit sign, and fire alarm detection. The next step involves gathering more sample data for minority classes like blocked exit violations to enhance detection performance.

6. Discussion

On the whole, the performance of this neural network appears quite satisfactory for such a 15-class problem defined on the customized dataset. Indeed, values of mAP@0.5 and F1 score of 97.8% and 0.95 respectively prove that YOLOv8 can be used successfully for solving the current problem due to its high level of accuracy and speed of making predictions. As far as major limitations detected within the framework of the project are concerned, the confusion matrix allows one to notice that the classes 'boxes blocking exit' and 'objects blocked emergency exit' tend to be misclassified mutually. Namely, these two classes are similar from a semantic perspective, i.e., they both represent an image of some physical objects obstructing an emergency exit. The difference lies in the nature of obstructions. As a result, these two classes can be merged into one called 'exit obstruction'.

The low AP of "person" class (0.936) as opposed to almost all other classes is due to the fact that there is a wide variety in the appearance of people in terms of scale, pose, and partial occlusion. This is an acknowledged problem in person detection tasks. One way to increase person detection accuracy would be to use a bigger backbone model, such as YOLOv8m and YOLOv8l.

The head/helmet distinction from a safety monitoring point of view is the most important issue. If the system were to confuse an unshielded head for being under a helmet, then the system might overlook that particular safety breach. The ideal approach would therefore be to set a lower confidence threshold when classifying the 'helmet' category of images.

It should also be mentioned that the data.yaml file uses the same train folder for training and validation. A separate test set can be more accurate when estimating generalization ability. This is something to consider in future development.

7. Ethical Reflection

This work was undertaken with thoughtful consideration of the ethical considerations of using AI for surveillance and computer vision. To ensure the system is fair, private, accountable, and transparent, a number of precautions were taken.

Privacy and Face Protection

One of the key ethical considerations for any safety monitoring system is the potential for identification of individuals. To overcome this, all human faces in the dataset were intentionally blurred before the dataset was used to train the model. This guarantees that the model never "sees" any identifiable facial features and therefore cannot be used as an identifier. In addition, the images in the dataset were obtained via Roboflow, a dataset provider specialising in computer vision training data, indicating that the images were already being used for this purpose, not simply downloaded randomly from the internet. Even if the face blurs were to be undone, the images are part of a training data set and not a privacy concern.

No Personal Data Collection

No personal data was gathered, stored or processed. It only recognises the classes of helmets, exit signs and fire alarms. No names, IDs, biometric information or geolocation data was collected throughout the pipeline. This is in line with data minimisation principles, which require data collection should be limited to the minimum necessary to achieve the task.

Privacy Concerns in AI Surveillance

Using computer vision systems in public or semi-public spaces like a university campus has justifiable concerns with over-surveillance. A safety-monitoring system could, in principle, be exploited for monitoring people's movement and activities. So, it is critical that if this system were to be deployed in practice, it would have a clear policy for its use, communication to the people being monitored, and restricted access to system output. This system should only be used for safety monitoring and not expanded further without review and approval by an ethics committee.

Responsible AI Usage

The system is meant to help with safety monitoring, not to make decisions or automate punishments. The model's findings should be inspected by a human before any action is taken. This supports the practice of human oversight in AI deployments to prevent overreliance on AI predictions that could adversely impact individuals. The limitations of the model, such as decreased accuracy on minority classes such as blocked exit violations, also highlight the importance of human review for action.

8. Real-World Implementation

For instance, the implementation of such a solution on the university premises will include hooking up the YOLOv8 algorithm, which has already been trained, with the university's CCTV cameras. The footage from each of the cameras will then be analyzed using a local inference engine or edge-based GPU processor. An NVIDIA Jetson AGX Orin, or any other equivalent edge computer, will have enough computing power to process YOLOv8's inference on 30 frames per second, with multiple cameras connected simultaneously, without having to access the internet

The detection results can then be fed into an application dashboard that will show real-time alerts in cases where a safety breach is detected, such as an obstacle in front of an emergency exit point, someone in a safety area but not wearing a helmet, and the blocking of a fire alarm system. Alerts can be set to activate only if the detection result has a minimum level of confidence and persists beyond a minimum period of time (for instance, at least 3 seconds).

There are several constraints that will have to be considered prior to implementation. Firstly, the training dataset of 1,672 images is somewhat limited in size compared to other detectors that use at least 5,000 images when classifying up to 15 different classes. The model will need to be tested with the data captured by the cameras deployed in the university, since the shift in domains (i.e., the differences between the camera's characteristics and those used to train the detector) might result in performance decrease. Additionally, performance drift, which happens over time as the environmental conditions change, should be accounted for. It is essential to conduct periodic evaluation and re-training of the model in order to achieve high accuracy. Lastly, all requirements concerning data protection according to the regulations of the UAE and the university's data governance strategy should be followed.

9. Conclusion

The project was able to showcase the entire pipeline from data collection to evaluation of a computer vision model for campus safety monitoring, which uses YOLOv8. Images were collected from Roboflow for our custom-built dataset comprising 1,672 images with annotations across 15 different safety-related objects, a YOLOv8m model was trained for 50 epochs with the help of a GPU, and it was tested based on metrics such as Precision, Recall, F1, and mAP@0.5. The best result we got was mAP@0.5=97.8%, peak F1 = 0.95.

The project validates that the current generation of single-shot detectors such as YOLOv8 provides an ideal compromise between accuracy and efficiency in real-world safety surveillance scenarios. Convergence was seamless, and the model demonstrates good transferability to new data as seen from the inference performance. Although there are still issues concerning the small dataset of some visual class pairs like “boxes blocking exit” and “objects blocked emergency exit,” they can be overcome through data augmentation and reclassification of classes.

In looking forward, some key areas where this project can be expanded include implementation of multiple object tracking (such as using SORT or DeepSORT) to track the objects and initiate sustained violation alerts, implementation of anomaly detection in order to detect abnormal occurrences that are not currently covered within the set of classes, and expanding on the classification schema to include more hazards related to campus well-being such as wet floor warning signs and fire extinguisher blocks.

Links:

GitHub Link: Hassan-Mujtabata/safety-detection-yolov8: YOLOv8m model for campus safety detection - BCS407 AI Project

Everything is explained in the readme

Dataset Link : https://huggingface.co/Hassanmujtabat/safety-detection-yolov8/tree/main

References

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-End Object Detection with Transformers. *ArXiv:2005.12872 [Cs]*. https://arxiv.org/abs/2005.12872

Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. *2014 IEEE Conference on Computer Vision and Pattern Recognition*, 580–587. https://doi.org/10.1109/cvpr.2014.81

Han, J., Liu, Y., Li, Z., Liu, Y., & Zhan, B. (2023). *Safety Helmet Detection Based on YOLOv5 Driven by Super-Resolution Reconstruction*. *23*(4), 1822–1822. https://doi.org/10.3390/s23041822

Jocher, G. (2020, May 18). *YOLOv8 Documentation*. Docs.ultralytics.com. https://docs.ultralytics.com/

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single Shot MultiBox Detector. *Computer Vision – ECCV 2016*, *9905*(5), 21–37. https://doi.org/10.1007/978-3-319-46448-0_2

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2015, June 8). *You Only Look Once: Unified, Real-Time Object Detection*. ArXiv.org. https://arxiv.org/abs/1506.02640

Ren, S., He, K., Girshick, R., & Sun, J. (2015, June 4). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. ArXiv.org. https://arxiv.org/abs/1506.01497

Zheng, H., Duan, J., Dong, Y., & Liu, Y. (2023). *Real-time fire detection algorithms running on small embedded devices based on MobileNetV3 and YOLOv4*. *19*(1). https://doi.org/10.1186/s42408-023-00189-0